Smart Beta Equity Portfolio Optimization

Complete Methodology, Data Pipeline, Modeling, Optimization, and Backtesting Workflow

1. Project Objective

The goal of this project is to build a complete smart beta portfolio construction pipeline that predicts next-month stock excess returns and then allocates capital across a selected set of stocks using mean-variance optimization (MVO). The strategy is evaluated with a rolling-window out-of-sample backtest to determine whether it delivers stronger risk-adjusted performance than a simple benchmark portfolio.

More specifically, the decision problem is:

At each monthly rebalancing date, how can we use historical factor information to predict next-month stock excess returns, select the most attractive investable stocks, and allocate portfolio weights in a way that maximizes risk-adjusted return?

The project therefore has two linked stages:

1. **Estimation modeling:** predict next-month stock excess returns using one traditional statistical model and one machine learning model.
2. **Decision optimization:** use the predicted returns together with an estimated covariance matrix to compute optimal portfolio weights.

The complete workflow is implemented under a rolling-window framework so that both the predictive model and the portfolio strategy are evaluated under true out-of-sample conditions.

2. Data Sources and Core Variables

2.1 CRSP Monthly Stock Data (`y_data`)

The stock-level monthly panel comes from CRSP. The main variables used in this project are:

- `PERMNO`: unique stock identifier
- `MthCalDt`: monthly calendar date
- `MthPrc`: month-end stock price

- `MthRet`: monthly stock return
- `MthVol`: monthly trading volume
- `ShrOut`: shares outstanding

These variables are sufficient to construct stock returns, price-based filters, a liquidity proxy, and historical return series for covariance estimation.

2.2 Fama–French Five-Factor Data (`x_data`)

The macro/style factor dataset includes:

- `Date`
- `Mkt-RF`
- `SMB`
- `HML`
- `RMW`
- `CMA`
- `RF`

These factors form the explanatory variables used in the return-prediction stage, while `RF` is also used to construct stock excess returns.

2.3 Target Variable

For stock i in month t, define the current-month excess return as:

$$ExcessRet_{i,t} = MthRet_{i,t} - RF_t. \tag{1}$$

The prediction target is the next-month excess return:

$$y_{i,t+1} = ExcessRet_{i,t+1}. \tag{2}$$

Operationally, this is created by grouping the panel by `PERMNO`, sorting observations by time, and shifting the excess return column forward by one month using `shift(-1)`.

2.4 Feature Vector

The baseline feature vector is the five-factor vector observed at month t:

$$\mathbf{X}_t = \left[(MKT - RF)_t,\ SMB_t,\ HML_t,\ RMW_t,\ CMA_t\right]. \tag{3}$$

Because the factor values are observed at the month level, all stocks in the same month share the same factor environment. The cross-sectional differentiation comes from the fact that different stocks respond differently during model estimation and portfolio selection.

3. Data Alignment and Cleaning Pipeline

3.1 Date Alignment

The CRSP monthly date `MthCalDt` is converted into an integer key in `YYYYMM` format:

$$DateKey_t = 100 \times Year(MthCalDt) + Month(MthCalDt). \tag{4}$$

The Fama–French variable `Date` is already provided in monthly `YYYYMM` format. These two keys are then used to merge the stock-level monthly data with the factor data.

3.2 Factor Scaling

The Fama–French factor values and the risk-free rate are typically stored in percentage units. To make them consistent with CRSP returns, all factor columns are divided by 100:

$$Factor_t^{scaled} = \frac{Factor_t}{100}. \tag{5}$$

This applies to:

$$Mkt - RF,\ SMB,\ HML,\ RMW,\ CMA,\ RF.$$

3.3 Price Cleaning

CRSP may record negative prices as a data sign convention. To avoid distortions in filtering and liquidity calculations, the price variable is cleaned as:

$$P_{i,t} = |MthPrc_{i,t}|. \tag{6}$$

In the code implementation, this is stored as `MthPrc_Abs`.

3.4 Merging the Two Datasets

After standardizing dates and factor scales, the stock panel and factor data are merged by month:

$$Panel_{i,t} = CRSP_{i,t} \bowtie FF5_t. \tag{7}$$

Each row in the merged dataset represents one stock-month observation that includes stock-level return and trading variables together with the factor environment observed in that month.

4. Universe Filtering and Investable Stock Pool Construction

The CRSP universe is too large and noisy to use directly in portfolio optimization. Therefore, we define a filtered investable universe before selection and weighting.

4.1 Missing-Value Filter

Observations are removed if any of the following variables are missing:

$$MthRet,\ P_{i,t},\ MthVol,\ Mkt-RF,\ SMB,\ HML,\ RMW,\ CMA,\ RF.$$

This ensures that every remaining stock-month can be used consistently for label creation, feature construction, and optimization.

4.2 Price Filter

To remove very low-price stocks, we keep only observations satisfying:

$$P_{i,t} \geq 5. \tag{8}$$

This step removes penny stocks, reduces extreme volatility, and helps prevent the optimizer from assigning unstable weights to highly speculative names.

4.3 Trading-Volume Filter

To remove illiquid or inactive stocks, we keep only observations satisfying:

$$MthVol_{i,t} > 0. \tag{9}$$

This helps make the portfolio more realistic from an implementation perspective.

4.4 Minimum History Filter

Because the project uses a 36-month rolling window for model estimation and covariance estimation, a stock is only eligible at month t if it has at least 36 valid months of prior history. Formally:

$$HistoryLen_{i,t} \geq 36. \tag{10}$$

This condition ensures that each eligible stock has enough history to support model estimation and risk estimation.

4.5 Liquidity Proxy for Candidate-Pool Ranking

After the basic investability filters, a liquidity measure is constructed as monthly dollar volume:

$$DollarVol_{i,t} = P_{i,t} \times MthVol_{i,t}. \tag{11}$$

This variable is later used to form the candidate pool from which the final portfolio holdings are chosen.

5. From the Full CRSP Universe to the Final Six Stocks

The project should not choose six stocks arbitrarily. Instead, the six final holdings are selected through a transparent, model-driven filtering and ranking process.

5.1 Selection Logic

The monthly stock-selection pipeline is:

Raw CRSP Universe → Filtered Investable Universe → Liquidity-Based Candidate Pool → Top 6 by Predicted E

5.2 Candidate Pool Formation

At each rebalancing month t, all stocks that pass the investability filters are ranked by liquidity using $DollarVol_{i,t}$. The top N stocks are retained as the candidate pool. A practical choice is:

$$CandidatePool_t = Top100(DollarVol_{i,t}). \tag{12}$$

Using the top 100 most liquid stocks each month balances realism, computational efficiency, and robustness.

5.3 Final Stock Selection

For every stock in the candidate pool, the estimation model produces a predicted next-month excess return:

$$\hat{y}_{i,t+1}. \tag{13}$$

The final six holdings are chosen as the top six stocks ranked by predicted return:

$$\mathcal{S}_t = Top6\left(\hat{y}_{i,t+1}\right). \tag{14}$$

5.4 Why Six Stocks?

Choosing six stocks is a practical trade-off:

- Too few holdings create excessive concentration risk.
- Too many holdings dilute the model's high-conviction ideas and make the optimization harder to interpret.
- Six holdings provide some diversification while keeping the portfolio focused and computationally manageable.

6. Estimation Modeling

The project uses one traditional statistical model and one machine learning model, as required by the class project guidelines.

6.1 Model 1: Ordinary Least Squares (OLS)

The OLS benchmark model is specified as:

$$y_{i,t+1} = \alpha + \beta_1(MKT - RF)_t + \beta_2 SMB_t + \beta_3 HML_t + \beta_4 RMW_t + \beta_5 CMA_t + \varepsilon_{i,t+1}. \tag{15}$$

Here,

$$y_{i,t+1} = ExcessRet_{i,t+1}. \tag{16}$$

This model provides a transparent linear baseline and makes it easy to interpret the directional effect of each factor on expected future excess returns.

6.2 Model 2: Random Forest Regressor

The machine learning model is a global random forest regressor trained on the same feature set. Its prediction can be written as:

$$\hat{y}_{i,t+1} = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{X}_t), \tag{17}$$

where $T_b(\cdot)$ is the b-th regression tree and B is the total number of trees in the forest.

The random forest is intended to capture non-linear relationships and interactions among the factor variables that may be missed by the linear OLS specification.

6.3 Model Inputs and Outputs

For both models:

- **Input features:**

$$\mathbf{X}_t = [(MKT - RF)_t, SMB_t, HML_t, RMW_t, CMA_t]$$

- **Target label:**

$$y_{i,t+1} = ExcessRet_{i,t+1}$$

- **Output:**

$$\hat{y}_{i,t+1} = \widehat{ExcessRet}_{i,t+1}$$

7. Rolling-Window Backtesting Framework

A rolling-window framework is used to generate true out-of-sample predictions and portfolio returns. This is critical for avoiding look-ahead bias and evaluating how the strategy would have behaved in a realistic monthly implementation setting.

7.1 Window Length

The rolling estimation window is fixed at 36 months:

$$L = 36 \text{ months}. \tag{18}$$

7.2 Monthly Rolling Logic

At each rebalancing month T:

1. use data from months $T - 35$ through T as the training sample,

2. predict excess returns for month $T+1$,
3. construct the portfolio for month $T+1$,
4. observe the realized returns in month $T+1$,
5. move the window forward by one month and repeat.

Symbolically,

$$[T-35, T] \to \text{predict } T+1, \tag{19}$$

and then

$$[T-34, T+1] \to \text{predict } T+2. \tag{20}$$

7.3 Why No Random Train-Test Split?

A random split is not appropriate because it would break the temporal ordering of financial data and could introduce look-ahead bias. All model training, prediction, and evaluation must preserve chronological order.

8. Detailed Monthly Workflow Inside Each Rolling Step

For each month T in the rolling backtest, the full implementation sequence is as follows.

8.1 Step 1: Build the Training Set

Extract all stock-month observations from the previous 36 months:

$$Train_T = \{(\mathbf{X}_{i,t}, y_{i,t+1}) : t \in [T-35, T]\}. \tag{21}$$

This training set contains the entire panel of eligible stock-month observations from the rolling window.

8.2 Step 2: Train the Predictive Models

Fit the OLS model and the random forest model using the same rolling training sample. This produces:

- OLS coefficient estimates $\hat{\alpha}, \hat{\beta}_1, \ldots, \hat{\beta}_5$
- a fitted random forest model for non-linear prediction

8.3 Step 3: Form the Month-T Candidate Pool

Within month T, keep all stocks that satisfy the investability filters and rank them by $DollarVol_{i,T}$. Then keep the top 100 names:

$$CandidatePool_T = Top100(DollarVol_{i,T}). \tag{22}$$

8.4 Step 4: Generate Predicted Returns

For each stock in the candidate pool, use the month-T factor values to predict next-month excess return.

For OLS:

$$\hat{y}^{OLS}_{i,T+1}. \tag{23}$$

For random forest:

$$\hat{y}^{RF}_{i,T+1}. \tag{24}$$

8.5 Step 5: Select the Six Final Holdings

Rank the candidate stocks by predicted excess return and select the top six:

$$\mathcal{S}^{model}_T = Top6\left(\hat{y}^{model}_{i,T+1}\right). \tag{25}$$

This step is repeated separately for the OLS-based strategy and the random-forest-based strategy.

8.6 Step 6: Estimate the Covariance Matrix

For the six selected stocks, extract their realized monthly returns over the past 36 months and compute the sample covariance matrix:

$$\Sigma_T = Cov(R_{T-35:T}). \tag{26}$$

In implementation, Σ_T is a 6×6 covariance matrix built from the historical return series of the selected stocks.

9. Decision Optimization with Mean-Variance Optimization

Once the six final holdings and their predicted returns are available, the portfolio weights are determined using MVO.

9.1 Expected Return Vector

The expected return vector for the selected stocks is:

$$\boldsymbol{\mu}_T = \begin{bmatrix} \hat{y}_{1,T+1} \\ \hat{y}_{2,T+1} \\ \vdots \\ \hat{y}_{6,T+1} \end{bmatrix}. \tag{27}$$

9.2 Risk Matrix

The corresponding covariance matrix is the historical estimate:

$$\Sigma_T. \tag{28}$$

9.3 Optimization Objective

Because the predictive models generate expected excess returns, the natural objective is to maximize the portfolio Sharpe ratio:

$$\max_{\mathbf{w}} \quad \frac{\mathbf{w}^\top \boldsymbol{\mu}_T}{\sqrt{\mathbf{w}^\top \Sigma_T \mathbf{w}}}. \tag{29}$$

9.4 Constraints

The portfolio is subject to the following constraints:

$$\sum_{i=1}^{6} w_i = 1 \qquad \text{(fully invested)} \tag{30}$$

$$w_i \geq 0, \quad i = 1, \ldots, 6 \qquad \text{(long-only)} \tag{31}$$

To improve stability, an optional upper bound can be imposed on each stock weight:

$$w_i \leq 0.40. \tag{32}$$

This prevents the optimizer from concentrating too much capital in a single stock.

9.5 Optimization Output

The final output of the optimization step is the portfolio weight vector:

$$\mathbf{w}_T^* = [w_{1,T}, w_{2,T}, \ldots, w_{6,T}]. \tag{33}$$

10. Realized Portfolio Return Calculation

After the portfolio is formed at month T, the realized out-of-sample portfolio return in month $T+1$ is computed using the actual stock returns in the next month:

$$R_{p,T+1} = \sum_{i=1}^{6} w_{i,T} \cdot MthRet_{i,T+1}. \tag{34}$$

For every rolling step, the implementation should store:

- the rebalance month,
- the six selected stocks,
- their predicted excess returns,
- their optimized weights,
- their realized next-month returns,
- the realized portfolio return.

11. Benchmark Construction

To evaluate whether optimization adds value beyond stock selection alone, the optimized portfolio is compared against an equal-weight benchmark built on the same six selected stocks.

11.1 Equal-Weight Benchmark

For the same six stocks selected in month T, assign equal weights:

$$w_i^{EW} = \frac{1}{6}. \tag{35}$$

The benchmark's realized return in month $T+1$ is:

$$R_{EW,T+1} = \frac{1}{6} \sum_{i=1}^{6} MthRet_{i,T+1}. \tag{36}$$

11.2 Interpretation

This benchmark helps answer whether the MVO layer improves portfolio performance beyond a simple equal allocation over the same model-selected stocks.

12. Evaluation Metrics

The project should evaluate both prediction quality and portfolio performance.

12.1 Prediction Metrics

12.1.1 Root Mean Squared Error (RMSE)

$$RMSE = \sqrt{\frac{1}{N}\sum_{j=1}^{N}(y_j - \hat{y}_j)^2}. \tag{37}$$

12.1.2 Mean Absolute Error (MAE)

$$MAE = \frac{1}{N}\sum_{j=1}^{N}|y_j - \hat{y}_j|. \tag{38}$$

Here,

$$y_j = ExcessRet_{i,t+1}, \qquad \hat{y}_j = \widehat{ExcessRet}_{i,t+1}.$$

These metrics compare the predictive accuracy of OLS and random forest on out-of-sample observations.

12.2 Portfolio Performance Metrics

Suppose the out-of-sample monthly portfolio return sequence is

$$R_{p,1}, R_{p,2}, \ldots, R_{p,T}.$$

12.2.1 Annualized Return

$$Annualized\ Return = \left(\prod_{t=1}^{T}(1 + R_{p,t})\right)^{12/T} - 1. \tag{39}$$

12.2.2 Annualized Volatility

$$Annualized\ Volatility = Std(R_{p,t}) \times \sqrt{12}. \tag{40}$$

12.2.3 Sharpe Ratio

$$Sharpe = \frac{\overline{R_{p,t} - RF_t}}{Std(R_{p,t} - RF_t)} \times \sqrt{12}. \tag{41}$$

12.2.4 Maximum Drawdown

Define cumulative wealth as:

$$Wealth_t = \prod_{s=1}^{t}(1 + R_{p,s}). \tag{42}$$

Define the running peak as:

$$Peak_t = \max_{1 \le s \le t} Wealth_s. \tag{43}$$

Then drawdown is:

$$Drawdown_t = \frac{Peak_t - Wealth_t}{Peak_t}, \tag{44}$$

and maximum drawdown is:

$$MDD = \max_t Drawdown_t. \tag{45}$$

13. Final Output Tables

The final report and presentation should include at least two key result tables.

13.1 Prediction Comparison Table

Model	RMSE	MAE
OLS	...	...
Random Forest	...	...

13.2 Portfolio Performance Comparison Table

Portfolio	Annualized Return	Sharpe Ratio	Max Drawdown
OLS + MVO	...	...	...
RF + MVO	...	...	...
Equal-Weight Benchmark	...	...	...

These two tables allow the team to compare the two modeling approaches and evaluate whether optimization improves realized portfolio performance.

14. Key Assumptions and Limitations

14.1 Stability of Factor-Return Relationships

A core assumption is that historical factor-return relationships remain informative about future returns:

$$\text{Historical factor-return relationships} \Rightarrow \text{future predictive signal.} \tag{46}$$

This assumption may weaken in changing market regimes, especially after major structural shifts. The rolling-window design partly addresses this by continuously re-estimating the model.

14.2 Historical Covariance as a Proxy for Future Risk

The optimization step assumes that the historical covariance matrix is a reasonable approximation to future risk:

$$\Sigma_{past} \approx \Sigma_{future}. \tag{47}$$

This is a standard but imperfect assumption in portfolio optimization.

14.3 Sensitivity of MVO to Input Error

MVO is well known to be sensitive to estimation error in both expected returns and covariance estimates. This project partially mitigates that issue by:

- limiting optimization to only six stocks,
- imposing long-only constraints,
- optionally imposing upper bounds on individual weights.

15. Implementation Checklist

The full coding workflow can be implemented in the following order.

Phase 1: Read and Align Data

1. Read CRSP monthly stock data.
2. Read Fama–French five-factor data.
3. Convert `MthCalDt` to datetime.
4. Create monthly key `DateKey = YYYYMM`.

5. Divide all factor columns and `RF` by 100.
6. Create `MthPrc_Abs = abs(MthPrc)`.
7. Merge the two datasets by `DateKey`.

Phase 2: Clean and Filter the Universe

8. Remove observations with missing key variables.
9. Keep only observations with $MthPrc_Abs \geq 5$.
10. Keep only observations with $MthVol > 0$.
11. Compute $DollarVol = MthPrc_Abs \times MthVol$.
12. For each month, retain only stocks with at least 36 months of valid history.

Phase 3: Construct Labels and Features

13. Compute $ExcessRet_{i,t} = MthRet_{i,t} - RF_t$.
14. Group by `PERMNO` and create the label $y_{i,t+1}$ using `shift(-1)`.
15. Set the feature vector to the Fama–French five factors.
16. Drop rows without a valid future label.

Phase 4: Build the Rolling Framework

17. Sort all available months.
18. Start the backtest from the first month with 36 months of prior history.
19. For each month T, define the estimation window $[T - 35, T]$.

Phase 5: Run the Monthly Loop

20. Build the rolling training sample.
21. Fit the OLS model.
22. Fit the random forest model.
23. Rank eligible month-T stocks by dollar volume and keep the top 100.
24. Predict next-month excess return for each candidate stock.

25. Rank by predicted return and select the top six stocks.
26. Use the past 36 months of realized returns for those six stocks to estimate the covariance matrix.
27. Solve the MVO problem for the optimized weight vector.
28. Read the actual next-month stock returns.
29. Compute the optimized portfolio return.
30. Compute the equal-weight benchmark return.
31. Save the results for that month.

Phase 6: Summarize Results

32. Concatenate all out-of-sample monthly returns.
33. Compute cumulative wealth curves.
34. Compute annualized return, Sharpe ratio, and maximum drawdown.
35. Compute out-of-sample RMSE and MAE for the two prediction models.
36. Prepare the final comparison tables and plots.

16. Conclusion

This project is designed as a complete decision-optimization pipeline rather than a standalone forecasting exercise. The process begins with CRSP stock data and Fama–French factors, constructs next-month excess-return labels, filters the stock universe to retain investable names, applies rolling-window prediction models, selects the six highest-conviction stocks each month, and then allocates capital through long-only mean-variance optimization. Performance is finally evaluated out of sample against an equal-weight benchmark using both predictive and portfolio-level metrics.

As a result, the project satisfies the core requirements of statistical estimation, machine learning, decision optimization, and robust out-of-sample backtesting, while also producing a workflow that is transparent enough to present clearly in the final project report and presentation.